Session 7 · Deck 7.B1

Redistribution prohibited without permission

PyTorch, Curve Fitting, and Yield Forecasts

Zhigang Feng

Thursday, November 5, 2026

© Zhigang Feng

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are

you are here: 7.B1 ▼

S1 ✓	S2 ✓	S3 ✓	S4 ✓	S5 ✓	S6 ✓	S7	S8	S9	S10
Sep 24	Oct 1	Oct 8	Oct 15	Oct 22	Oct 29	Nov 5	Nov 12	Nov 19	Dec 3
Intro	HA · OLG	Num. DP	Numerics	Numerics	HA classic	ML	RL	Text	Agents
Growth	Policy	Python	I	II	The wall	Deep nets	HA + DL	LLMs	Projects
trunk	branches	the machine · classical methods			the wall	new tools		new data	colleagues

Build it on paper → solve it on the machine → push it to the frontier with AI

Block A — learning a function: 7.A1 the learning problem and the neuron · 7.A2 activations and approximation · 7.A3 backpropagation and training. **Block B — using it: 7.B1** PyTorch and a forecast · 7.B2 the growth model · 7.B3 architecture and shape · 7.B4 capacity and order.

© Zhiyang Fan. Redistribution prohibited without permission

Roadmap: Three Parts

Block A on paper; now in code. First where the answer is known, then on a real series.

Part	Topic	The question it answers
1	PyTorch in five ideas	What is the library, and how do you know the loop is right?
2	A curve whose answer is known	Can a network find a law nobody wrote into the code, and how do you score it?
3	Next month's 10-year yield	Does the same loop run end to end on a real economic series, honestly scored?

© Zheng Feng Redistribution prohibited without permission

Idea

Everything in 7.A3 is four library calls. Only the model and the loss change between applications; the five-line loop does not.

Two pieces of substance, two of bookkeeping.

tensors
arrays of numbers, as in NumPy

autograd
a record of every operation, so exact derivatives come back

nn.Module
a class that holds a network's parameters

optim
gradient-descent variants: SGD, Adam

- A scientific-computing library that became popular for deep learning, not the reverse.
- The first two pieces are the substance: write any scalar as a chain of array operations and its gradient is free.
- The last two are bookkeeping for parameters and update rules.
- **Everything in 7.A3 is four library calls. Today we make them.**

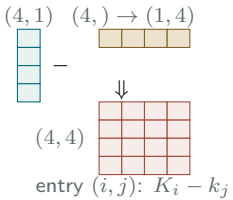
Redistribution prohibited without permission
© Zhigang Feng

A. Paszke et al., "PyTorch: an imperative style, high-performance deep learning library", *NeurIPS* 32, 2019. 3.B3 introduced it.

1 · Tensors: Shapes, Types, and the Broadcasting Bug

Keep n observations of d variables as (n, d), and print the shape.

```
k = torch.linspace(0.5, 2.0, 4)      # shape (4,)
K = k.reshape(-1, 1)                 # shape (4, 1): four observations, one feature
print(k.shape, K.shape, K.dtype)     # torch.Size([4]) torch.Size([4, 1]) torch.float32
print((K - k).shape)                 # torch.Size([4, 4])   <- not what you meant
```

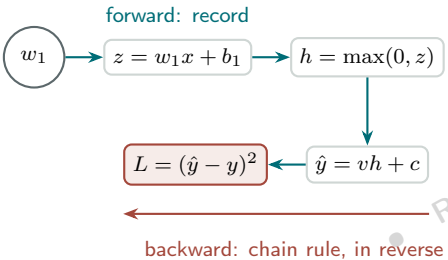


- A **tensor** is an array with a **shape** (4 rows, 1 column) and a **dtype** (32-bit floats).
- **Broadcasting** lines shapes up from the right, so a $(4,)$ is read as one row, $(1, 4)$.
- Each size-1 axis is copied to match: the column across, the row down. Result: every K_i minus every k_j, silently.
- **Keep n observations of d variables as (n, d), and print the shape before you trust the number.**

Lab 7, block A1. 3.B2 met the same rule in NumPy.

© Zhigang Feng
Redistribution prohibited without permission

You write the forward pass only; the library runs the chain rule backwards.



- Mark a tensor `requires_grad=True` and PyTorch records every operation it enters: the **computation tape**.
- `backward()` walks the tape in reverse, one chain-rule step per operation: an exact derivative, not a finite difference (3.B3's floor does not apply).
- This is **autograd**: the chain rule, applied by the library — 7.A3's backward pass.
- **You write the forward pass only.**

© Zhigang Feng Redistribution prohibited without permission

1· backward(): One Step You Can Check by Hand

7.A3's four-parameter ReLU example, run by the library.

```
x, y = 2.0, 3.0
w1, b1, v, c = (torch.tensor(a, requires_grad=True) for a in (0.5, -0.1, 1.2, 0.3))
z = w1 * x + b1; h = torch.relu(z); yhat = v * h + c; L = (yhat - y) ** 2
L.backward()
```

```
# forward : z = 0.9000, h = 0.9000, yhat = 1.3800, L = 2.6244
# backward: dL/dv = -2.9160, dL/dc = -3.2400, dL/dw1 = -7.7760, dL/db1 = -3.8880
# update  : w1 -> 0.5778, v -> 1.2292;  loss 2.6244 -> 1.7488     (eta = 0.01)
```

- A four-parameter ReLU network on one observation. Each derivative is one chain-rule product, and the lab asserts every number against autograd — and against 7.A3's hand calculation.
- A second hidden unit with $z = -0.8 < 0$ receives a gradient of exactly 0.0: no signal, no learning.
- **If your hand derivative and autograd disagree, the hand derivative is wrong.**

© Zhiang Feng · Redistribution prohibited without permission

Lab 7, block A2; example from Miao & Feng, *AI for Economists*, §9.3.

1 · The Training Loop Is Always the Same Five Lines

Forward, loss, clear, backward, update. Learn the loop once.

```
model = nn.Linear(2, 1)
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
for step in range(300):
    prediction = model(X)                    # 1 forward
    loss = ((prediction - y) ** 2).mean()    # 2 loss
    optimizer.zero_grad()                    # 3 clear the old gradients
    loss.backward()                          # 4 backward
    optimizer.step()                         # 5 update
```

- Forward, loss, clear, backward, update. A language model with billions of parameters is trained by these five lines.
- Only line 1 (the model) and line 2 (the loss) change between applications.
- Fit a curve: L is the squared prediction error. Solve a model: L is the squared Euler residual, which is 7.B2.
- **Once derivatives are free, one algorithm covers both. Learn the loop once.**

© Lingang Feng · Redistribution prohibited without permission

1 · A Network Is a Class

The class is the specification. Read it as you would read a regression equation.

```
class SimpleNet(nn.Module):
    def __init__(self, hidden=32, act=nn.ReLU):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(1, hidden), act(), nn.Linear(hidden, 1))
    def forward(self, x):
        return self.net(x)
```

- `nn.Module` is PyTorch's base class for anything with parameters. `__init__` declares the layers; `forward` says how an input flows through them — 3.B2's model-as-a-class.
- `nn.Linear(1, hidden)` is $Wx + b$ with W of shape hidden $\times$ 1; `nn.Sequential` composes layers in order.
- This is $f_\theta(x) = W_2\,\sigma(W_1 x + b_1) + b_2$ from 7.A2, with 32 hidden units: 97 parameters.
- **The class is the specification. Read it as you would read a regression equation.**

© Zhiliang Fang · Redistribution prohibited without permission

Lab 7, block A4.

1 · Optimizers, zero_grad, and Seeds

Forgetting `zero_grad()` raises no error. It trains the wrong model.

```
torch.manual_seed(1); net = SimpleNet()              # same seed, same initial draw
optimizer = torch.optim.Adam(net.parameters(), lr=3e-3, weight_decay=weight_decay)
optimizer.zero_grad()                                # 3 clear the old gradients
loss = loss_fn(net(X_tr), y_tr); loss.backward(); optimizer.step()
net.eval()                                           # evaluation mode
with torch.no_grad():                                # no tape: nothing to differentiate
    val = float(loss_fn(net(X_va), y_va))
```

- `parameters()` hands every W and b to the **optimizer**. `weight_decay` is an L2 penalty on the weights: ridge regression, in one argument.
- Gradients *accumulate* across calls to `backward()`: without `zero_grad()` you descend on the sum of every gradient so far.
- Initial weights are a random draw and a **seed** fixes it. `eval()` switches layers such as dropout; `no_grad()` turns the tape off when you only want predictions.
- **Forgetting `zero_grad()` raises no error. It trains the wrong model.**

© Zhirang Feng · Redistribution prohibited without permission

Lab 7, blocks A4 and C.

1 · Start at Version 0, Where You Know the Answer

A linear "network" trained on squared error must land on OLS. It does.

```
torch.manual_seed(0)
X = torch.randn(200, 2)
y = 1.0 + X @ torch.tensor([[2.0], [-0.5]]) + 0.1 * torch.randn(200, 1)
#  ... the five-line loop on nn.Linear(2, 1), 300 steps ...
# OLS    : [ 1.0073  1.9972 -0.5023]
# learned: [ 1.0073  1.9972 -0.5023]
assert np.allclose(ols, learned, atol=1e-3)
```

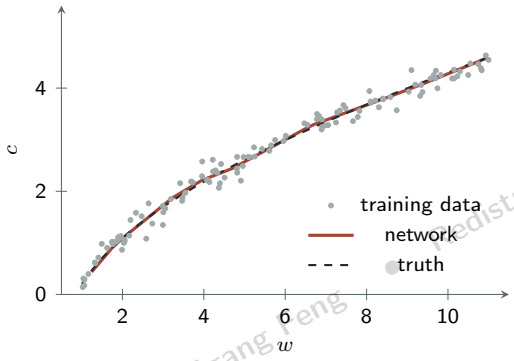
- A linear "network" trained on squared error must land on the OLS coefficients. It does, to four decimals.
- If it had not, the bug would be in the loop, and you would know before touching a real problem.
- The same habit in 7.B2: solve the growth model first where the policy is known in closed form.
- **Version 0 of any project is the case with a known answer** — HW1's A2 unit test at $\delta = 1$.

© Zhenting Hong Redistribution prohibited without permission

Lab 7, block A3.

Fit: a Network Finds $c = f(w)$

The functional form was never specified, and the test error sits at the noise floor.



- A hidden law, $c = \ln w + 0.2\,w + \varepsilon$ on $w \in [1, 11]$, noise variance 0.0225; 300 training and 100 test observations.
- 32 ReLU units, 1,500 Adam steps. The functional form was never specified.
- Mean squared error: 0.0231 in training, 0.0229 on the test data.

© Zhigang Feng · redistribution prohibited without permission

Lab 7, block B.

2 · The Code, Block 1: Make the Data and Split It

We wrote the law ourselves, so we know the right answer before fitting anything.

```
torch.manual_seed(42)
w = torch.rand(400, 1) * 10 + 1
c = torch.log(w) + 0.2 * w + 0.15 * torch.randn(400, 1)
idx = torch.randperm(400); tr, te = idx[:300], idx[300:]
mu, sd = w[tr].mean(), w[tr].std()
```

- Line 2: 400 draws of w, uniform on $[1, 11]$, shape $(400, 1)$: n observations of one variable.
- Line 3: the hidden law plus noise with standard deviation 0.15, so the noise variance is $0.15^2 = 0.0225$.
- Line 4: shuffle the row numbers; the first 300 train, the last 100 test. A random split is fine for a cross-section. The yields, a time series, will be split by date.
- Line 5: the scaler's mean and standard deviation come from the training rows only.
- **We wrote the law ourselves, so we know the right answer before fitting anything.**

© Zhigang Peng · Redistribution prohibited without permission

The Code, Block 2: the Network and the Training Loop

Nowhere does the code say $\ln w$.

```
def fit_curve(hidden=32, act=nn.ReLU, epochs=1500, seed=0):
    torch.manual_seed(seed)
    net = SimpleNet(hidden, act)
    opt = torch.optim.Adam(net.parameters(), lr=1e-2)
    for _ in range(epochs):
        loss = ((net((w[tr] - mu) / sd) - c[tr]) ** 2).mean()
        opt.zero_grad(); loss.backward(); opt.step()
    net.eval()
    return net
```

- Lines 2–4: fix the initial draw; build `SimpleNet` (1 input, 32 ReLU units, 1 output); hand its Ws and bs to Adam.
- Line 6: forward on the standardised inputs, then the mean squared error against c.
- Line 7: clear, backward, update. The five-line loop, 1,500 times on all 300 rows.
- **Nowhere does the code say $\ln w$. The functional form is never written down.**

© Zhigang Feng · Redistribution prohibited without permission

Lab 7, block B.

The Code, Block 3: Score It on Both Samples

The test number is the one to report: those 100 rows played no part in fitting.

```
net = fit_curve()
with torch.no_grad():
    mse = {name: ((net((w[i] - mu) / sd) - c[i]) ** 2).mean().item()
           for name, i in (("train", tr), ("test", te))}
print({k: round(v, 4) for k, v in mse.items()}, "| noise variance = 0.0225")
# {'train': 0.0231, 'test': 0.0229} | noise variance = 0.0225
```

- Line 2: `no_grad()` turns the tape off. We only want predictions now.
- Lines 3–4: the same formula on each sample, picked by its row numbers. The test rows are standardised with the training `mu` and `sd`.
- The noise variance, 0.0225, is the error even the true law would make. No model can beat it on average.
- Training and test errors are close to each other and to that floor.
- **The test number is the one to report: those 100 rows played no part in fitting.**

© Zhigang Feng · Redistribution prohibited without permission

Lab 7, block B; the plotting lines are left out.

The Data: Five FRED Series, Monthly

Every input is something a forecaster could have known at the end of month t.

FRED series	What it is	How it enters
DGS10, DGS3MO	10-year and 3-month Treasury yields, daily	last trading day of each month
FEDFUNDS	effective federal funds rate, monthly	value for month $t-1$
UNRATE	unemployment rate, monthly	value for month $t-1$
CPIAUCSL	consumer price index, monthly	12-month inflation, month $t-1$

© Zhicang Feng • Redistribution prohibited without permission

- Downloaded once, on 20 September 2026: 439 monthly rows, the last for August 2026.
- Yields are market prices, seen at month-end t. Macro series are published with a lag, so the value used at t is the one for month $t-1$.
- **Every input is something a forecaster could have known at the end of month t.**

Federal Reserve Bank of St. Louis, FRED, fred.stlouisfed.org; Lab 7, block C.

At the end of month t, predict the 10-year yield at the end of month $t+1$.



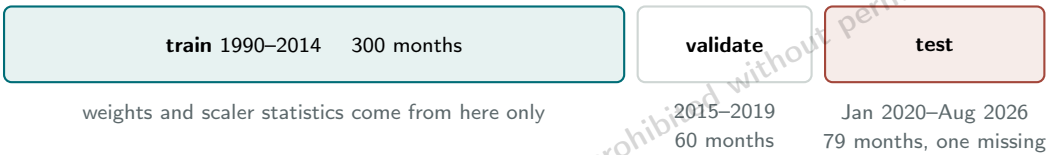
FRED series DGS10, month-end, downloaded 20 Sep 2026.

- At the end of month t, predict the 10-year yield at the end of month $t+1$.
- Inputs: the 10-year yield and the 10-year minus 3-month spread at t; funds rate, unemployment and inflation for $t-1$.
- The model: a small network that predicts the *change*; forecast = this month's level + change.
- **The aim is not to beat the bond market, but to fit a real series end to end.**

© Zhigang Feng Redistribution prohibited without permission

Splitting the Data by Date

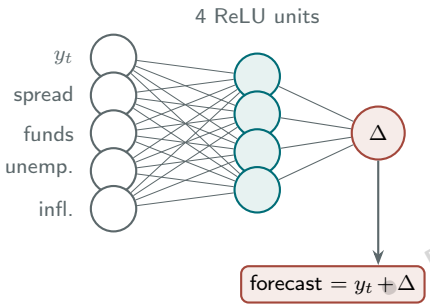
Anything estimated is estimated on the training window.



© Zhirang Fang · Redistribution prohibited without permission

- Split by the *date of the label*, in order. No shuffle: next month is not a fresh draw.
- **Training** data fit the weights; **validation** data choose the width and when to stop; **test** data are used once, at the end.
- Inputs are standardised with the mean and standard deviation of 1990–2014 only.
- 79, not 80: the December 2025 label is missing, because FRED has no October 2025 CPI or unemployment value.
- **Anything estimated is estimated on the training window.**

Small on purpose: 300 training months cannot pin down many parameters.

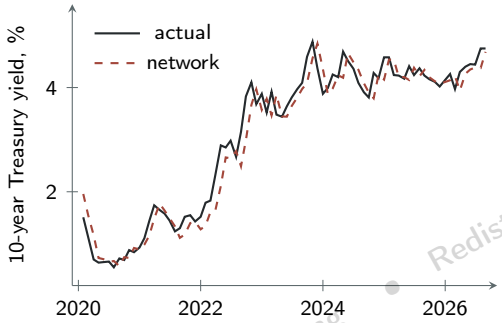


- Five standardised inputs, one hidden layer; the output is the *change*, so zero means "no change".
- Width picked on validation from 4, 8, 16 (MSE 0.035, 0.047, 0.040): 4 units, 29 parameters.
- Full-batch Adam, step 0.003, weight decay 0.001 (ridge); stop after 40 epochs without a validation gain, keep the best.
- Ten seeds give test RMSEs of 0.27 to 0.36 points; the next frames average them.
- **Small on purpose: 300 training months cannot pin down many parameters.**

© Zhigang Feng — Redistribution not allowed without permission

Lab 7, block C.

The training loop from Part 1, unchanged, runs on a real economic series.



- The 79 test months, never seen in training. Red: the forecast, averaged over ten seeds.
- It tracks the path through the 2020 low and the 2022 rise.
- Most of the tracking comes from starting at this month's level: the learned changes are small, 0.06 points on average against actual moves of 0.21.
- **The training loop from earlier in this deck, unchanged, runs on a real economic series.**

© Zhigang Feng
Redistribution prohibited without permission

Lab 7, block C; FRED series DGS10, month-end, downloaded 20 Sep 2026.

The Score, Against a Random Walk

One information set, one split, one metric — and the random walk is not beaten.

Test, 79 months	RMSE	MAE
random walk, $\hat{y}_{t+1} = y_t$	0.266	0.210
AR(1)	0.265	0.209
OLS, five inputs	0.286	0.225
network, mean of ten seeds	0.308	0.241
network, average forecast	0.296	0.229

Percentage points of yield. One seed alone ranges from 0.266 to 0.359.

- Month-ahead yields are close to a martingale, so the random walk is the bar — and nothing here clears it.
- Averaging the ten forecasts beats the average of their ten scores: 0.296 against 0.308.
- The seed spread (0.09) is larger than the gap to OLS: report the spread with the score.
- **The point of the exercise was the honest comparison, not the win.**

Lab 7, block C. The frozen forecasts are kept for Session 9's text signal.

© Zhigang Feng · Redistribution prohibited without permission

Takeaways

- **Part 1.** PyTorch is tensors and autograd, plus bookkeeping. The five-line loop — forward, loss, clear, backward, update — is checked twice before it is trusted: `backward()` against 7.A3's hand derivatives, and a linear network against OLS.
- **Part 2.** A 32-unit network finds $c = \ln w + 0.2w$ without being told the form: test error 0.0229 against a noise floor of 0.0225.
- **Part 3.** The same loop on five FRED series, split by date and scaled on the training window: the forecast tracks the yield, mostly by starting at this month's level, and does not beat a random walk.

© Zhigang Feng • Redistribution prohibited without permission

Bridge: A Loss with No Data in It

Here the label was data: a consumption level, next month's yield. A model has no such labels. Nobody hands us the right k' at each k.

What a model does hand us is the condition a right answer must satisfy. Make its residual the prediction error and zero the label, and the five-line loop solves the model.

Next: 7.B2

The growth model solved as supervised learning on its Euler equation: feasibility by construction, where to sample, and tests on states the network never saw — against time iteration (3.A3) and the closed form.

© Zhigang Feng Redistribution prohibited without permission